HANDS-ON LAB 03 GUIDE & COPY-PASTE PROMPTS • OMNICHANNEL RETAIL EDITION (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 03 (EN)

Lab 03: Retail MCP Server, Deterministic Margin Hooks & Enterprise Security Governance

Persona: Technical & Commercial Track | **Prerequisites**: Lab 02 Completed

Language / Idioma: English | Español • **Prompt Playbook:** PROMPTS.md | PROMPTS.es.md • **PDF:** EN (Dark) | EN (Light) | ES (Dark) | ES (Light)

1. Retail Scenario & Business Context

Even after encoding rules and skills (Lab 02), two major operational challenges remain in an enterprise retail organization:

1. **Live System Grounding via Model Context Protocol (MCP)**:

Neither engineers nor commercial category managers want to copy-paste raw SQL queries or CSV exports into chat windows. Instead, Antigravity should query the live **`data/retail_pulse.db`** SQLite database directly through structured **Model Context Protocol (MCP)** tools to inspect True ATP across nodes, audit breaching vendor OTIF contracts, simulate promotional margins before launch, and list VIP orders at risk of stockout.

2. **Deterministic Guardrails & Enterprise Governance**:

Instructions in **`.agents/rules/`** guide the LLM's reasoning, but mission-critical compliance—such as blocking destructive SQL mutations (**`DROP`**/**`TRUNCATE`**/**`DELETE`** on **`wms_inventory`**, **`omnichannel_orders`**, or **`vendor_otif_contracts`**) or blocking someone from hardcoding **`margin_floor_pct = 0.0`** / **`DISABLE_MARGIN_FLOOR`**—requires **deterministic enforcement** that executes outside the LLM's probabilistic output, alongside **Google Cloud / Gemini Enterprise governance** and **Antigravity Client Sandbox & Permission controls**.

2. Learning Objectives

In this lab, you will:

1. Complete the **Retail Pulse FastMCP Server** (**`mcp_server/retail_mcp.py`**) by implementing three high-impact MCP tools alongside the starter **`query_true_atp_inventory`** tool:

- **`audit_vendor_otif_contracts(only_breaching: bool = True)`**: Audits supplier On-Time In-Full (OTIF) contracts and calculates deterministic chargeback penalties (**`$98,250.00`** across 6 breaching suppliers).
- **`simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "")`**: Simulates stacking promotional codes against a SKU and reports whether the stacked discounts trigger the 15% gross margin

floor clamp (**`margin_floor_clamped`**).

- **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND")`**: Queries **`omnichannel_orders`** + **`wms_inventory`** + **`sku_catalog`** and evaluates **`evaluate_location_atp(..., required_qty=row["qty"])["oversell_risk"]`** to list VIP orders at risk of stockout and their total Customer Lifetime Value (**`total_vip_ltv_at_risk_usd`**).

2. Register the local MCP server in **`.agents/mcp_config.json`** so Antigravity discovers and invokes all 4 retail MCP tools automatically (**`"command": "./.venv/bin/python", "args": ["mcp_server/retail_mcp.py"]`**).
3. Configure deterministic **`PreToolUse`** and **`PostToolUse`** lifecycle hooks in **`.agents/hooks.json`** backed by **`scripts/margin_guard_hook.py`** (**`--phase pre`** and **`--phase post`**) that intercept destructive SQL or margin-floor bypasses (**`exit code 2`** with **`{"decision": "deny", ...}`**) and write an immutable local audit trail to **`data/logs/hook_compliance_audit.jsonl`**.
4. Walk through **Google Cloud & Gemini Enterprise Governance** and **Antigravity Client & Workspace Security Controls** for enterprise retail deployments.

3. Step-by-Step Instructions

Step 1: Initialize the Lab 03 Starter State

Run the Lab 03 installer to ensure Labs 01 and 02 solutions are active and load the Lab 03 starter files:

```
./labs/lab-03-mcp-and-hooks/install.sh
```

Verify that the Lab 03 test suite currently fails on the unimplemented MCP tools (**`TODO_LAB_03`** stubs) and missing **`.agents/mcp_config.json`** / **`.agents/hooks.json`** entries:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Step 2: Inspect the Starter MCP Server (`mcp_server/retail_mcp.py`)

Open **`mcp_server/retail_mcp.py`**. Notice that one MCP tool is already implemented:

- **`query_true_atp_inventory(sku: str, location_id: str | None = None, required_qty: int = 1) -> str`**

However, three tools are currently **`TODO_LAB_03`** stubs:

- **`audit_vendor_otif_contracts(only_breaching: bool = True) -> str`**
- **`simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "") -> str`**
- **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str`**

Step 3: Implement the Remaining MCP Tools & Register `.agents/mcp_config.json`

Open **`labs/lab-03-mcp-and-hooks/PROMPTS.md`** and use the **Implementation Prompts** (or **Inline Edit `Cmd+I` / `Ctrl+I`**) to:

1. Implement **`audit_vendor_otif_contracts(only_breaching: bool = True) -> str`** in **`mcp_server/retail_mcp.py`** by delegating to **`audit_all_vendor_otif_contracts(only_breaching=only_breaching)`**.

2. Implement **`simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "") -> str`** in **`mcp_server/retail_mcp.py`** by splitting **`promo_codes_csv`** into a list of uppercase promo codes and calling **`calculate_promotional_order_pricing(sku=sku, qty=qty, promo_codes=codes)`**.
3. Implement **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str`** in **`mcp_server/retail_mcp.py`** by joining **`omnichannel_orders`**, **`wms_inventory`**, and **`sku_catalog`**, evaluating **`evaluate_location_atp(..., required_qty=int(row["qty"]))`**, filtering to orders where **`atp_eval["oversell_risk"]`** is **`True`** (or **`status == "AT_RISK_STOCKOUT"`**), and returning **`loyalty_tier_filter`**, **`at_risk_order_count`**, **`total_vip_ltv_at_risk_usd`**, and **`orders`**.
4. Update **`.agents/mcp_config.json`** to register the **`retail-pulse-mcp`** server via **`"command": "./.venv/bin/python", "args": ["mcp_server/retail_mcp.py"]`**.

Step 4: Configure the Deterministic `PreToolUse` & `PostToolUse` Margin Guard Hooks (`.agents/hooks.json`)

Inspect **`scripts/margin_guard_hook.py`**. This script reads the tool call payload from **`stdin`**:

- During **`--phase pre`** (**`PreToolUse`**), if a command or file edit attempts destructive SQL (**`DROP TABLE`**, **`TRUNCATE`**, or **`DELETE FROM wms_inventory|omnichannel_orders|vendor_otif_contracts`**) or attempts to bypass the 15% margin floor (**`margin_floor_pct = 0.0`** or **`DISABLE_MARGIN_FLOOR`**), the hook exits with code **`2`** and outputs **`{"decision": "deny", "reason": "..."}`**.
- Every hook evaluation (**`allow`** or **`deny`**) is appended as a structured JSON event to **`data/logs/hook_compliance_audit.jsonl`**.

Configure **`.agents/hooks.json`** to wire **`./.venv/bin/python scripts/margin_guard_hook.py --phase pre`** to **`PreToolUse`** (**`run_command|execute_command|bash|write_file|edit_file`**) and **`./.venv/bin/python scripts/margin_guard_hook.py --phase post`** to **`PostToolUse`**.

Test the hook directly from the terminal to verify both the block behavior and the audit log entry:

```
echo '{"tool_name":"edit_file","tool_input":{"path":"src/services/promo_pricing.py","con \
  tent":"margin_floor_pct = 0.0 # DISABLE_MARGIN_FLOOR"}}' | ./.venv/bin/python scripts/ \
  margin_guard_hook.py --phase pre
cat data/logs/hook_compliance_audit.jsonl
```

4. Enterprise Security, Privacy & Governance Deep-Dive

When deploying Antigravity across a retail enterprise (merchandising, supply chain, store operations, and platform engineering), security architects govern the platform at **two complementary layers**:

```
+-------------------------------------------------------------------------------+
| LAYER 1: Google Cloud & Gemini Enterprise Governance (Cloud Admin Plane)       |
|  - IAM & Identity: roles/cloudaicompanion.user via Cloud Identity / Corporate SSO |
|  - Cloud Audit Logs: cloudaicompanion.googleapis.com in Google Cloud Logging   |
|  - Data Privacy: Zero Data Retention & No Model Training on customer prompts/code |
|  - Network Perimeter: VPC Service Controls (VPC-SC) & Regional Endpoints       |
|  - IP Protection: Google Cloud Indemnification & Citation Filters              |
+-------------------------------------------------------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------------------+
| LAYER 2: Antigravity Client & Workspace Security Controls (Local Execution Plane) |
|  - Tool Execution Policy: strict | request-review | proceed-in-sandbox         |
|  - Terminal Sandbox & Network Isolation: OS-level process & filesystem sandboxing |
```

```
|  - Non-Workspace File Access: Restrict read/write strictly to repo boundary       |
|  - Command Allowlist / Denylist & Browser Allowlist                               |
|  - Deterministic Lifecycle Hooks: .agents/hooks.json -> compliance audit JSONL    |
+-----------------------------------------------------------------------------------+
```

Layer 1: Google Cloud & Gemini Enterprise Governance

Control Area	Google Cloud / Gemini Enterprise Capability	Retail Enterprise Use Case
IAM & Seat Management	Assign **`roles/cloudaicompanion.user`** via Google Cloud IAM integrated with **Cloud Identity / Okta / Azure AD (Entra ID)** SSO groups.	Grant access separately to **`retail-platform-eng@`** and **`retail-commercial-ops@`** with centralized onboarding/offboarding.
Cloud Audit Logging	Enable **Admin Activity** and **Data Access** audit logs for **`cloudaicompanion.googleapis.com`** in **Google Cloud Logging**, and route via Log Sinks to BigQuery or Chronicle SIEM.	Maintain a tamper-evident enterprise audit trail of who invoked Gemini Enterprise, from which IP/device, and when.
Data Privacy & Zero Training	Customer prompts, workspace context, SQL schemas, and generated code are **never used to train shared foundation models** and adhere to strict zero-retention guarantees outside your tenant boundary.	Safely analyze proprietary vendor contracts (**`data/exports/vendor_contracts.csv`**), wholesale unit costs, and VIP customer LTV data.
VPC Service Controls (VPC-SC)	Place **`cloudaicompanion.googleapis.com`** inside a **VPC Service Controls perimeter** alongside BigQuery, Cloud SQL, and Secret Manager, with regional endpoint pinning.	Prevent data exfiltration from corporate workstations or unauthorized external networks.

Layer 2: Antigravity Client & Workspace Security Controls

Client Setting / Hook	Configuration Options	Recommended Retail Production Setting
Tool Execution Policy	**`strict`**, **`request-review`**, **`proceed-in-sandbox`**, **`always-proceed`**	Use **`proceed-in-sandbox`** or **`request-review`** so mutating shell commands require either OS sandboxing or human approval.
Terminal Sandbox & Network Policy	OS-level sandbox (**`sandbox-exec`** on macOS / namespaces on Linux) + Internet Access Toggle	Enable Terminal Sandbox and restrict external network calls so agents can only reach approved internal endpoints.
Non-Workspace File Access	Block or require confirmation for file reads/writes outside the workspace root	**Block** non-workspace access so agents cannot read **`~/.ssh`**, **`~/.aws`**, or unrelated directories on the host machine.
Command Allowlist / Denylist	Prefix allowlist (e.g., **`pytest`**, **`.venv/bin/python`**) and denylist (**`rm -rf`**, **`curl`**, **`nc`**)	Allowlist **`.venv/bin/python -m pytest`** for autonomous TDD while blocking destructive shell or network exfiltration commands.

Client Setting / Hook	Configuration Options	Recommended Retail Production Setting
Browser Allowlist	Domain allowlist for the `/browser` subagent	Restrict `/browser` strictly to `http://localhost:8002` (`http://127.0.0.1:8002`) and approved internal staging dashboards.
Deterministic Lifecycle Hooks	`.agents/hooks.json` (`PreToolUse`, `PostToolUse`)	Enforce commercial and security invariants (such as `scripts/margin_guard_hook.py`) and log every check to `data/logs/hook_compliance_audit.jsonl`.

5. Verification Checklist

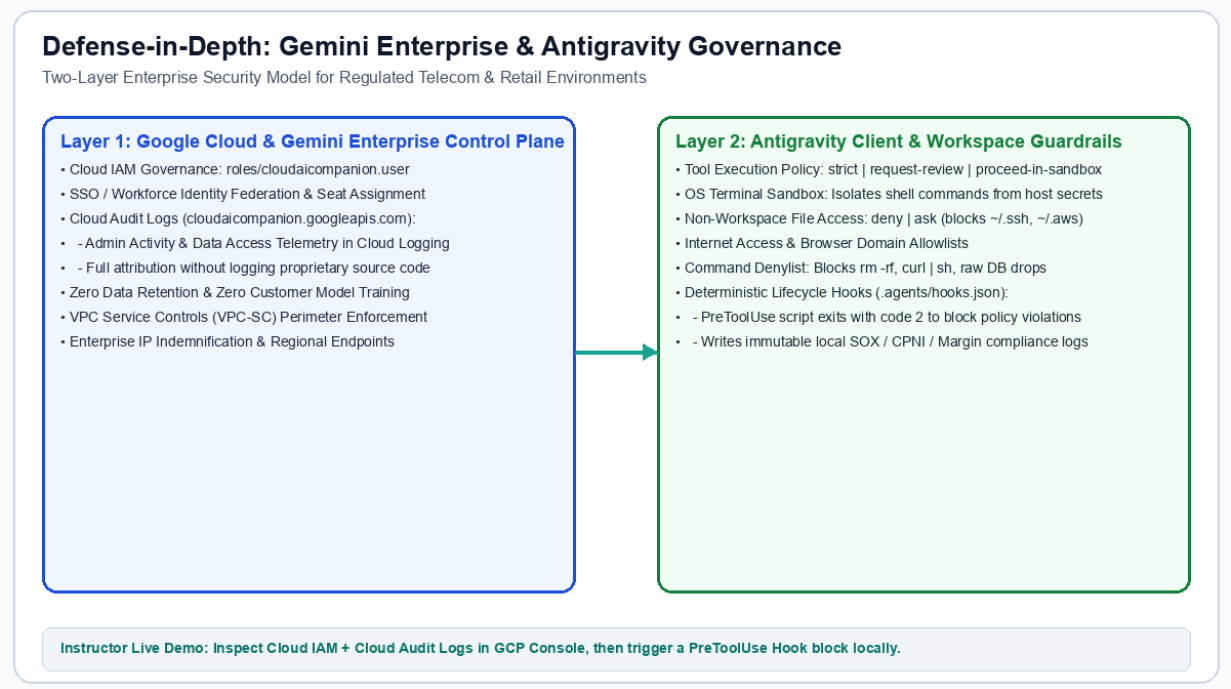
Run the Lab 03 verification script to confirm your MCP tools, `.agents/mcp_config.json`, and `.agents/hooks.json` pass all automated tests:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Expected output:

- `tests/test_lab03_mcp_hooks.py` passes **3/3 Lab 03 tests** (**13/13 cumulative tests** across Labs 01–03):
- `test_mcp_config_and_hooks_json_are_valid`
- `test_retail_mcp_tools_return_accurate_live_sqlite_data`
- `test_margin_guard_hook_blocks_unsafe_mutations_and_allows_safe_commands`

Need to jump straight to the verified state? Run `./labs/lab-03-mcp-and-hooks/apply_solution.sh`.



LAB 03 COPY-PASTE ANTIGRAVITY PROMPTS (EN)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Lab 03 Prompt Playbook: Retail MCP Server & Deterministic Margin Hooks

Language / Idioma: English | Español • **Lab Guide:** README.md | README.es.md • **PDF:** EN | ES

Copy and paste these prompts into **Google Antigravity** as you progress through Lab 03. Each section includes prompts tailored for both the **Technical Track** and the **Commercial / Merchandising Track**.

Phase 1: Discovery & Architecture Inspection

Prompt 1A — Technical Track: Inspect the Starter MCP Server & Hook Script

```
Read @mcp_server/retail_mcp.py, @scripts/margin_guard_hook.py, and @tests/test_lab03_mcp \
  _hooks.py.
Summarize:
1. How the starter MCP tool (`query_true_atp_inventory`) connects to `data/retail_pulse. \
  db` and our service layer.
2. What exact JSON fields `audit_vendor_otif_contracts`, `simulate_promo_margin_guard`,  \
  and `list_at_risk_vip_orders` must return to pass `tests/test_lab03_mcp_hooks.py`.
3. How `scripts/margin_guard_hook.py` inspects `--phase pre` (`PreToolUse`) and `--phase \
   post` (`PostToolUse`) payloads on stdin and writes compliance events to `data/logs/ho \
  ok_compliance_audit.jsonl`.
```

Prompt 1B — Commercial / Operations Track: Understand Live MCP Grounding vs. Deterministic Hooks

```
Read @mcp_server/retail_mcp.py and @scripts/margin_guard_hook.py. Explain in plain retai \
  l terms:
1. How the 4 MCP tools let a Merchandising or Supply Chain manager ask live questions ab \
  out True ATP stock, breaching vendor OTIF contracts, promo margin impact, and VIP orde \
  rs at risk without writing SQL.
2. Why we pair our `.agents/rules/retail-standards.md` guidelines with a deterministic ` \
  PreToolUse` hook (`scripts/margin_guard_hook.py`) that blocks anyone from turning off  \
  the 15% margin floor or running destructive SQL on `wms_inventory` and `omnichannel_or \
  ders`.
```

Phase 2: Planning (`/plan`)

Prompt 2 — Create an Implementation Plan for MCP Tools & Lifecycle Hooks

```
/plan Design the implementation for Lab 03 so that all tests in @tests/test_lab03_mcp_ho \
  oks.py pass:
1. Implement `audit_vendor_otif_contracts(only_breaching: bool = True) -> str`, `simulat \
  e_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "") -> str`, and ` \
  list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str` in @mcp_server/retail_m \
  cp.py.
2. Update `.agents/mcp_config.json` registering `retail-pulse-mcp` using `"command": "./ \
```

```
   .venv/bin/python"` and `"args": ["mcp_server/retail_mcp.py"]`.
3. Update `.agents/hooks.json` registering `./.venv/bin/python scripts/margin_guard_hook \
   .py --phase pre` on `PreToolUse` (`run_command|execute_command|bash|write_file|edit_fi \
   le`) and `./.venv/bin/python scripts/margin_guard_hook.py --phase post` on `PostToolUs \
   e`.
```

Phase 3: Inline Edit (`Cmd+I` / `Ctrl+I`)

Open **`mcp_server/retail_mcp.py`**, highlight the **`audit_vendor_otif_contracts`**, **`simulate_promo_margin_guard`**, and **`list_at_risk_vip_orders`** stub functions, press **`Cmd+I`** (macOS) or **`Ctrl+I`** (Windows/Linux), and enter:

```
Implement `audit_vendor_otif_contracts` using `audit_all_vendor_otif_contracts(only_brea \
  ching=only_breaching)`. Implement `simulate_promo_margin_guard` by parsing `promo_code \
  s_csv` into a list of uppercase codes and calling `calculate_promotional_order_pricing \
  (sku=sku, qty=qty, promo_codes=codes)`. Implement `list_at_risk_vip_orders` by joining \
   `omnichannel_orders`, `wms_inventory`, and `sku_catalog`, evaluating `evaluate_locati \
  on_atp` for each order, and returning JSON with `loyalty_tier_filter`, `at_risk_order_ \
  count`, `total_vip_ltv_at_risk_usd`, and `orders`.
```

Phase 4: Full Implementation Prompts

Prompt 4A — Implement `mcp_server/retail_mcp.py`, `.agents/mcp_config.json`, and `.agents/hooks.json`

```
Complete Lab 03 by performing the following changes:
1. In @mcp_server/retail_mcp.py:
   - Implement `audit_vendor_otif_contracts(only_breaching: bool = True) -> str` by retu \
  rning `json.dumps(audit_all_vendor_otif_contracts(only_breaching=only_breaching), inde \
  nt=2)`.
   - Implement `simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str \
   = "") -> str` by parsing `promo_codes_csv` into uppercase promo codes and returning ` \
  json.dumps(calculate_promotional_order_pricing(sku=sku, qty=qty, promo_codes=codes), i \
  ndent=2)`.
   - Implement `list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str` by queryi \
  ng `omnichannel_orders` joined with `wms_inventory` and `sku_catalog`, filtering to or \
  ders where `evaluate_location_atp(..., required_qty=int(row["qty"]))["oversell_risk"]` \
   is True (or `status == 'AT_RISK_STOCKOUT'`), and returning JSON with `loyalty_tier_fi \
  lter`, `at_risk_order_count`, `total_vip_ltv_at_risk_usd`, and `orders`.
2. Update `.agents/mcp_config.json` with a `mcpServers` entry for `retail-pulse-mcp` run \
  ning `"command": "./.venv/bin/python"` and `"args": ["mcp_server/retail_mcp.py"]`.
3. Update `.agents/hooks.json` with `PreToolUse` (`./.venv/bin/python scripts/margin_gua \
  rd_hook.py --phase pre`) and `PostToolUse` (`./.venv/bin/python scripts/margin_guard_h \
  ook.py --phase post`).
```

Phase 5: Verification & Live MCP Queries

Prompt 5A — Run Automated Verification (`verify.sh`)

```
Run `./labs/lab-03-mcp-and-hooks/verify.sh` and confirm all 13 cumulative tests across L \
  abs 01-03 pass (including all 3 tests in `tests/test_lab03_mcp_hooks.py`), and show th \
  e latest entry in `data/logs/hook_compliance_audit.jsonl`.
```

Prompt 5B — Commercial Track: Query the Retail MCP Tools for Executive Insights

```
Using the functions in @mcp_server/retail_mcp.py, answer these three commercial question \
  s:
1. If we stack promos `VIP20,FLASH15,CLEARANCE10` on `SKU-OLED-65` (qty=1), does it trig \
  ger the 15% margin floor clamp (`margin_floor_clamped`), and what is the clamped unit  \
  price?
2. How many `DIAMOND` VIP orders are currently at risk of stockout, and what is the tota \
  l Customer LTV exposed (`total_vip_ltv_at_risk_usd`)?
3. Which 6 breaching vendors fail their contracted OTIF SLA, and what is their combined  \
  chargeback penalty in USD (`total_chargeback_usd`)?
```

Phase 6: Stretch Challenge — Test the Margin Guard Hook Under Adversarial Prompting

Prompt 6 — Attempt to Bypass the 15% Margin Floor (Observe Hook Blocking)

```
Simulate a request from an unauthorized campaign manager: test running `scripts/margin_g \
  uard_hook.py --phase pre` against a tool payload that tries to edit `src/services/prom \
  o_pricing.py` with `margin_floor_pct = 0.0 # DISABLE_MARGIN_FLOOR`, and another payloa \
  d that tries to run `DROP TABLE wms_inventory;`. Show the hook exit codes (`2`), JSON  \
  `deny` decisions, and the audit entries recorded in `data/logs/hook_compliance_audit.j \
  sonl`.
```